Question1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Answer:

```
import java.io.*;
import java.util.*;

class Farmer {
String name, fruit, city;
int quantity;

Farmer(String name, String fruit, String city, int quantity) {
this.name = name;
this.fruit = fruit;
this.city = city;
this.quantity = quantity;
}
}

public class FarmerDataProgram {
public static void main(String[] args) throws Exception {

Scanner sc = new Scanner(System.in);
Farmer[] f = new Farmer[10];

// Input from user
for (int i = 0; i < 10; i++) {
System.out.println("Enter details of Farmer " + (i + 1));

System.out.print("Name: ");
String name = sc.nextLine();

System.out.print("Fruit: ");
String fruit = sc.nextLine();

System.out.print("City: ");
String city = sc.nextLine();
```

```
System.out.print("Quantity: ");
int quantity = Integer.parseInt(sc.nextLine());

f[i] = new Farmer(name, fruit, city, quantity);
}

// File writing
FileWriter fw = new FileWriter("farmers.txt");

fw.write("Before Sorting:\n");
fw.write("Name\tFruit\tCity\tQuantity\n");

for (Farmer x : f) {
fw.write(x.name + "\t" + x.fruit + "\t" + x.city + "\t" + x.quantity + "\n");
}

// Sorting
Arrays.sort(f, (a, b) -> a.quantity - b.quantity);

fw.write("\nAfter Sorting:\n");
fw.write("Name\tFruit\tCity\tQuantity\n");

for (Farmer x : f) {
fw.write(x.name + "\t" + x.fruit + "\t" + x.city + "\t" + x.quantity + "\n");
}

fw.close();

// Search
System.out.print("\nEnter farmer name to search: ");
String search = sc.nextLine();

boolean found = false;

for (Farmer x : f) {
if (x.name.equalsIgnoreCase(search)) {
System.out.println("Found:");
System.out.println(x.name + " " + x.fruit + " " + x.city + " " + x.quantity);
found = true;
break;
```

```
}
}

if (!found) {
System.out.println("Farmer not found");
}

sc.close();
}
}
```

OUTPUT:

```
 Enter details of Farmer 1
Name: vijay
Fruit: mango
City: Bilaspur
Quantity: 60

Enter details of Farmer 2
Name: rahul
Fruit: apple
City: Raipur
Quantity: 45

Enter details of Farmer 3
Name: amit
Fruit: banana
City: Durg
Quantity: 80

Enter details of Farmer 4
Name: suraj
Fruit: orange
City: Korba
Quantity: 55

Enter details of Farmer 5
Name: ramesh
Fruit: grapes
```

City: Jagdalpur
Quantity: 70

Enter details of Farmer 6
Name: mohit
Fruit: guava
City: Ambikapur
Quantity: 40

Enter details of Farmer 7
Name: deepak
Fruit: pineapple
City: Raigarh
Quantity: 65

Enter details of Farmer 8
Name: arjun
Fruit: papaya
City: Bhilai
Quantity: 50

Enter details of Farmer 9
Name: sumit
Fruit: watermelon
City: Kawardha
Quantity: 75

Enter details of Farmer 10
Name: ankit
Fruit: litchi
City: Bilaspur
Quantity: 35

create a file farmer.text ,this file have this content

Before Sorting:

Name	Fruit	City	Quantity
vijay	mango	Bilaspur	60
rahul	apple	Raipur	45
amit	banana	Durg	80
suraj	orange	Korba	55

ramesh grapes Jagdalpur 70
mohit guava Ambikapur 40
deepak pineapple Raigarh 65
arjun papaya Bhilai 50
sumit watermelon Kawardha 75
ankit litchi Bilaspur 35

After Sorting:
Name Fruit City Quantity
ankit litchi Bilaspur 35
mohit guava Ambikapur 40
rahul apple Raipur 45
arjun papaya Bhilai 50
suraj orange Korba 55
vijay mango Bilaspur 60
deepak pineapple Raigarh 65
ramesh grapes Jagdalpur 70
sumit watermelon Kawardha 75
amit banana Durg 80

Search output: ***Enter farmer name to search: vijay***

Found:
vijay mango Bilaspur 60

Question 2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

Answer:

```
import java.io.*;
import java.util.*;

class Fruit {
String name;
String quality;
```

```
int quantity;

Fruit(String name, String quality, int quantity) {
this.name = name;
this.quality = quality;
this.quantity = quantity;
}
}

public class FruitDataProgram {
public static void main(String[] args) throws Exception {

Scanner sc = new Scanner(System.in);
Fruit[] f = new Fruit[3];
String[] fruits = {"Mango", "Apple", "Papaya"};

// Input
for (int i = 0; i < 3; i++) {
System.out.println("Enter details of " + fruits[i]);

System.out.print("Quantity: ");
int qty = sc.nextInt();
sc.nextLine();

// Condition
if (qty > 1000) {
qty = qty / 2;
}

System.out.print("Quality (good/bad/average): ");
String quality = sc.nextLine();

f[i] = new Fruit(fruits[i], quality, qty);
}

// Save to Notepad
FileWriter fw = new FileWriter("fruits.txt");

fw.write("Fruit\tQuality\tQuantity\n");

for (Fruit x : f) {
```

```
fw.write(x.name + "\t" + x.quality + "\t" + x.quantity + "\n");
}

fw.close();

// Search
sc.nextLine();
System.out.print("\nEnter fruit name to search: ");
String sName = sc.nextLine();

System.out.print("Enter quality to search: ");
String sQuality = sc.nextLine();

boolean found = false;

for (Fruit x : f) {
if (x.name.equalsIgnoreCase(sName) &&
x.quality.equalsIgnoreCase(sQuality)) {

System.out.println("Found:");
System.out.println(x.name + " " + x.quality + " " + x.quantity);
found = true;
}
}

if (!found) {
System.out.println("Not Found");
}

sc.close();
}
}
```

output:

Enter details of Mango

Quantity: 100

Quality (good/bad/average): good

Enter details of Apple

Quantity: 100

Quality (good/bad/average): average

Enter details of Papaya

Quantity: 200

Quality (good/bad/average): good

Enter fruit name to search: mango

Enter quality to search: good

Found: Mango good 100

Queastion.3

There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view.

Search for a given fruit in the available orchards after taking input from the user & print the tree.

Answer:

```
import java.util.*;

class Orchard {

    String name;

    String fruit;

    String quality;


    Orchard(String name, String fruit, String quality) {

        this.name = name;

        this.fruit = fruit;

        this.quality = quality;

    }
```

```
}

public class OrchardTree {
    public static void main(String[] args) {

        // Data (given in question)
        Orchard[] orchards = {
            new Orchard("A", "Apple", "good"),
            new Orchard("B", "Litchi", "good"),
            new Orchard("C", "Mango", "good"),
            new Orchard("D", "Apple", "bad")
        };

        // Print Tree View
        System.out.println("Tree View of Orchards:");
        System.out.println("Orchards");
        for (Orchard o : orchards) {
            System.out.println("  ├── Orchard " + o.name);
            System.out.println(" |     ├── Fruit: " + o.fruit);
            System.out.println(" |     └── Quality: " + o.quality);
        }

        // Search
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit to search: ");
        String search = sc.nextLine();

        boolean found = false;
```

```
        for (Orchard o : orchards) {
            if (o.fruit.equalsIgnoreCase(search)) {
                System.out.println("\nFound in Orchard " + o.name +
                        " (Quality: " + o.quality + ")");
                found = true;
            }
        }

        if (!found) {
            System.out.println("\nFruit not found in any orchard");
        }

        sc.close();
    }
}
```

Output:

Tree View of Orchards:

```
Orchards
 ├── Orchard A
 |    ├── Fruit: Apple
 |    └── Quality: good
 ├── Orchard B
 |    ├── Fruit: Litchi
 |    └── Quality: good
 ├── Orchard C
 |    ├── Fruit: Mango
```

```
|   └── Quality: good
├── Orchard D
|   ├── Fruit: Apple
|   └── Quality: bad
```

Enter fruit to search: Apple

Found in Orchard A (Quality: good)

Found in Orchard D (Quality: bad)

Question4. Take a map provided as a graph & display all the subtrees of connected points.

Answer:

```
import java.util.*;

public class GraphSubtrees {

static void dfs(int node, boolean[] visited, List<List<Integer>> graph,
List<Integer> component) {
visited[node] = true;
component.add(node);

for (int neigh : graph.get(node)) {
if (!visited[neigh]) {
dfs(neigh, visited, graph, component);
}
}
}

public static void main(String[] args) {
```

```
int n = 7; // number of nodes (0 to 6)

// Graph (Adjacency List)
List<List<Integer>> graph = new ArrayList<>();

for (int i = 0; i < n; i++) {
graph.add(new ArrayList<>());
}

// Connections (edges)
graph.get(0).add(1);
graph.get(1).add(0);

graph.get(1).add(2);
graph.get(2).add(1);

graph.get(3).add(4);
graph.get(4).add(3);

graph.get(5).add(6);
graph.get(6).add(5);

boolean[] visited = new boolean[n];

System.out.println("Subtrees (Connected Components):");

// Find all components
for (int i = 0; i < n; i++) {
if (!visited[i]) {
List<Integer> component = new ArrayList<>();
dfs(i, visited, graph, component);

System.out.println(component);
}
}
```

```
}
}
```

Output:

Subtrees (Connected Components):

[0, 1, 2]

[3, 4]

[5, 6]

Question 5. Enter entry point & goal point & produce the matching subtree of the graph.

Answer:

```
import java.util.*;

public class GraphPath {

    static boolean dfs(int node, int goal, boolean[] visited,
                       List<List<Integer>> graph, List<Integer> path) {

        visited[node] = true;
        path.add(node);

        if (node == goal) {
            return true;
        }

        for (int neigh : graph.get(node)) {
            if (!visited[neigh]) {
                if (dfs(neigh, goal, visited, graph, path)) {
                    return true;
                }
            }
        }

        // backtrack
        path.remove(path.size() - 1);
        return false;
```

```
}

public static void main(String[] args) {

    int n = 7;

    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        graph.add(new ArrayList<>());
    }

    // Graph edges
    graph.get(0).add(1);
    graph.get(1).add(0);

    graph.get(1).add(2);
    graph.get(2).add(1);

    graph.get(2).add(3);
    graph.get(3).add(2);

    graph.get(3).add(4);
    graph.get(4).add(3);

    graph.get(4).add(5);
    graph.get(5).add(4);

    graph.get(5).add(6);
    graph.get(6).add(5);

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter entry point: ");
    int start = sc.nextInt();

    System.out.print("Enter goal point: ");
    int goal = sc.nextInt();

    boolean[] visited = new boolean[n];
    List<Integer> path = new ArrayList<>();
```

```
        boolean found = dfs(start, goal, visited, graph, path);

        if (found) {
            System.out.println("Matching Subtree (Path): " + path);
        } else {
            System.out.println("No path found");
        }

        sc.close();
    }
}
```

Output:

Case 1

Enter entry point: 0
Enter goal point: 4

Matching Subtree (Path): [0, 1, 2, 3, 4]

Case 2

Enter entry point: 2
Enter goal point: 6

Matching Subtree (Path): [2, 3, 4, 5, 6]

Case 3

Enter entry point: 0
Enter goal point: 10

No path found

Question6: Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Answer:

```
import java.util.*;

class Pair {
    int node, weight;

    Pair(int node, int weight) {
        this.node = node;
        this.weight = weight;
    }
}

public class ShortestPath {

    public static void main(String[] args) {

        int n = 6; // nodes 0 to 5

        List<List<Pair>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Weighted edges
        graph.get(0).add(new Pair(1, 4));
        graph.get(0).add(new Pair(2, 2));

        graph.get(1).add(new Pair(2, 5));
        graph.get(1).add(new Pair(3, 10));

        graph.get(2).add(new Pair(4, 3));

        graph.get(4).add(new Pair(3, 4));

        graph.get(3).add(new Pair(5, 11));

        graph.get(4).add(new Pair(5, 2));

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter start node: ");
        int start = sc.nextInt();
```

```
System.out.print("Enter end node: ");
int end = sc.nextInt();

int[] dist = new int[n];
int[] parent = new int[n];

Arrays.fill(dist, Integer.MAX_VALUE);
Arrays.fill(parent, -1);

dist[start] = 0;

PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.weight - b.weight);
pq.add(new Pair(start, 0));

while (!pq.isEmpty()) {
    Pair curr = pq.poll();

    for (Pair neigh : graph.get(curr.node)) {
        if (dist[curr.node] + neigh.weight < dist[neigh.node]) {
            dist[neigh.node] = dist[curr.node] + neigh.weight;
            parent[neigh.node] = curr.node;
            pq.add(new Pair(neigh.node, dist[neigh.node]));
        }
    }
}

// Reconstruct path
List<Integer> path = new ArrayList<>();
int temp = end;

while (temp != -1) {
    path.add(temp);
    temp = parent[temp];
}

Collections.reverse(path);

if (dist[end] == Integer.MAX_VALUE) {
    System.out.println("No path found");
} else {
```

```
            System.out.println("Shortest Path: " + path);
            System.out.println("Total Cost: " + dist[end]);
        }

        sc.close();
    }
}
```

Output:

Case 1

```
Enter start node: 0
Enter end node: 5

Shortest Path: [0, 2, 4, 5]
Total Cost: 7
```

Case 2

```
Enter start node: 1
Enter end node: 5

Shortest Path: [1, 2, 4, 5]
Total Cost: 10
```

Case 3

```
Enter start node: 5
Enter end node: 0

No path found
```

Question7. Write a program to demonstrate the use of all keywords made for exception handling.

Answer:

```
import java.util.*;

public class ExceptionDemo {
```

```
// throws keyword
static void checkAge(int age) throws Exception {
    if (age < 18) {
        // throw keyword
        throw new Exception("Not eligible to vote");
    } else {
        System.out.println("Eligible to vote");
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    try {
        System.out.print("Enter age: ");
        int age = sc.nextInt();

        int arr[] = {10, 20, 30};

        // Arithmetic Exception
        int result = 10 / (age - age);

        // Array Exception
        System.out.println(arr[5]);

        // method call
        checkAge(age);

    }
    catch (ArithmeticException e) {
        System.out.println("Arithmetic Exception: Division by zero");
    }
    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array Index Exception");
    }
    catch (Exception e) {
        System.out.println("General Exception: " + e.getMessage());
    }
    finally {
```

```
        System.out.println("Finally block always executes");
    }

    sc.close();
  }
}
```

Output:

Case 1 (Arithmetic Exception)

Enter age: 20
Arithmetic Exception: Division by zero
Finally block always executes

Case 2 (Array Exception)

Enter age: 1
Array Index Exception
Finally block always executes

Case 3 (Custom Exception - throw)

Enter age: 15
General Exception: Not eligible to vote
Finally block always executes

Question 8: Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Answer:

```
import java.util.*;

// Custom Exception
class NoMoneyException extends Exception {
  public NoMoneyException(String msg) {
    super(msg);
  }
```

```
}

class BankAccount {
   int balance;

   BankAccount(int balance) {
      this.balance = balance;
   }

   // withdraw method
   void withdraw(int amount) throws NoMoneyException {
      if (amount > balance) {
         throw new NoMoneyException("Insufficient Balance!");
      } else {
         balance -= amount;
         System.out.println("Withdrawal Successful");
         System.out.println("Remaining Balance: " + balance);
      }
   }
}

public class BankDemo {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      BankAccount acc = new BankAccount(5000);

      try {
         System.out.print("Enter amount to withdraw: ");
         int amt = sc.nextInt();

         acc.withdraw(amt);
      }
      catch (NoMoneyException e) {
         System.out.println("Exception: " + e.getMessage());
      }
      finally {
         System.out.println("Transaction Ended");
      }
```

```
        sc.close();
    }
}
```

Output:

Case 1 (Valid Withdrawal)

Enter amount to withdraw: 2000
Withdrawal Successful
Remaining Balance: 3000
Transaction Ended

Case 2 (Exception Case)

Enter amount to withdraw: 7000
Exception: Insufficient Balance!
Transaction Ended

Question 9: Write a program which enables to use a platform by a train one at a time.

Answer:

```
class Platform {

    // synchronized method → only one train at a time
    synchronized void usePlatform(String trainName) {

        System.out.println(trainName + " is entering the platform");

        try {
            Thread.sleep(2000); // simulate time on platform
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(trainName + " is leaving the platform\n");
    }
}
```

```
class Train extends Thread {
   Platform p;
   String name;

   Train(Platform p, String name) {
      this.p = p;
      this.name = name;
   }

   public void run() {
      p.usePlatform(name);
   }
}

public class RailwayDemo {
   public static void main(String[] args) {

      Platform p = new Platform();

      Train t1 = new Train(p, "Train A");
      Train t2 = new Train(p, "Train B");
      Train t3 = new Train(p, "Train C");

      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output:

```
Train A is entering the platform
Train A is leaving the platform

Train B is entering the platform
Train B is leaving the platform

Train C is entering the platform
Train C is leaving the platform
```

Question10: Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

Answer:

```
class Restaurant {

    String order;
    boolean available = false; // order ready hai ya nahi

    // Waiter places order
    synchronized void placeOrder(String order) {

        while (available) {
            try {
                wait(); // wait until chef finishes
            } catch (InterruptedException e) {}
        }

        this.order = order;
        System.out.println("Waiter placed order: " + order);

        available = true;
        notify(); // chef ko signal
    }

    // Chef prepares order
    synchronized void prepareOrder() {

        while (!available) {
            try {
                wait(); // wait until order arrives
            } catch (InterruptedException e) {}
        }

        System.out.println("Chef prepared: " + order);

        available = false;
```

```
        notify(); // waiter ko signal
    }
}

class Waiter extends Thread {
    Restaurant r;

    Waiter(Restaurant r) {
        this.r = r;
    }

    public void run() {
        String orders[] = {"Pizza", "Burger", "Pasta"};

        for (String o : orders) {
            r.placeOrder(o);
        }
    }
}

class Chef extends Thread {
    Restaurant r;

    Chef(Restaurant r) {
        this.r = r;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            r.prepareOrder();
        }
    }
}

public class RestaurantDemo {
    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        Waiter w = new Waiter(r);
        Chef c = new Chef(r);
```

```
        w.start();
        c.start();
    }
}
```

Output

```
Waiter placed order: Pizza
Chef prepared: Pizza
Waiter placed order: Burger
Chef prepared: Burger
Waiter placed order: Pasta
Chef prepared: Pasta
```